

AULA 2 · 18 de agosto · U1, Linguagens e Paradigmas

Paradigmas de programação

Três formas de organizar o mesmo programa, e o critério para escolher entre elas

CODING: LINGUAGENS E TÉCNICAS

Prof. Dr. Sebastião Rogério · Faculdade SENAC Pernambuco, ADS 2026.2



**O mesmo problema, resolvido de três maneiras.
Todas corretas.**

Um paradigma não é uma questão de gosto nem de moda. É uma decisão sobre onde colocar o estado do programa, e essa decisão define o que será fácil e o que será difícil dali em diante.

Onde a disciplina está

A aula anterior tratou de diferenças entre linguagens: tipagem, execução e memória.

Aquele era um olhar de fora. A pergunta era como uma linguagem se comporta, e a resposta ajudava a escolher qual delas usar. Os três eixos voltam na próxima aula, com mais profundidade e com um laboratório de medição.

Hoje a pergunta muda de nível. Escolhida a linguagem, resta decidir como organizar o programa dentro dela. E essa segunda decisão pesa mais no dia a dia do que a primeira, porque uma equipe troca de linguagem raramente e escolhe como estruturar código todos os dias.



Uma observação sobre a sequência

Paradigma não depende de linguagem. Os três estilos desta aula podem ser escritos em Python, em JavaScript ou em Java, e é justamente por isso que a discussão vem antes da unidade de orientação a objetos.

O que é um paradigma

Um paradigma de programação é um conjunto de ideias sobre como um programa deve ser estruturado.

Ele responde a perguntas anteriores à sintaxe: onde fica o estado do sistema, quem tem permissão para alterá-lo, como o programa é dividido em partes, e o que essas partes trocam entre si.

Uma linguagem costuma favorecer um paradigma, mas quase nenhuma linguagem moderna obriga a um só. Python, JavaScript, Java e C# permitem escrever nos três estilos que esta aula apresenta. A escolha, portanto, é do programador, e precisa ser justificada.



Por que essa discussão não é acadêmica

Código escrito em um paradigma e mantido por alguém que pensa em outro é uma das causas mais comuns de sistema difícil de alterar. O paradigma é uma convenção de equipe antes de ser uma preferência pessoal.

Os três estilos desta aula

1

Imperativo e procedural

O programa é uma sequência de comandos que altera o estado passo a passo.

A pergunta central: o que a máquina deve fazer, e em que ordem?

2

Funcional

O programa é uma composição de funções que transformam dados sem alterá-los no lugar.

A pergunta central: que transformação leva da entrada ao resultado?

3

Orientado a objetos

O programa é um conjunto de objetos que guardam o próprio estado e expõem o que sabem fazer.

A pergunta central: quem é responsável por quê?

Existem outros, fora do escopo desta aula

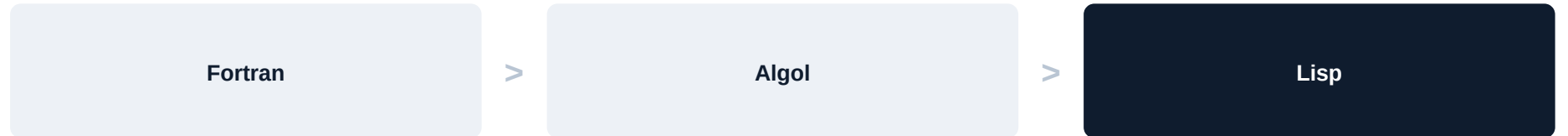
Lógico, como Prolog, e orientado a eventos, como boa parte do código de interface. Os três acima cobrem o que a disciplina vai usar.

De onde veio cada estilo

Cada paradigma surgiu como resposta a um limite prático do anterior.

Anos 1950 e 60

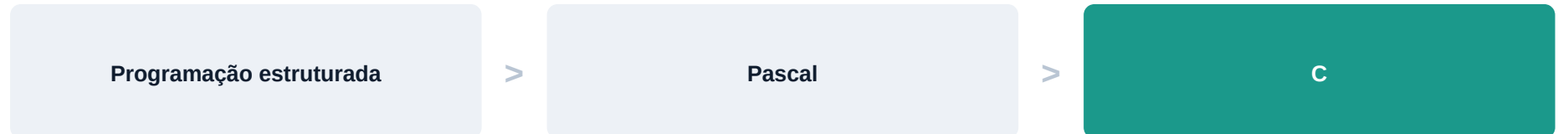
o começo



O estilo imperativo nasce colado ao funcionamento da máquina. Lisp aparece já em 1958 com as ideias funcionais, vindo da lógica matemática e não da engenharia elétrica.

Anos 1970

a crise do software



Projetos passam a falhar por complexidade, não por hardware. A resposta é disciplinar o fluxo do programa e eliminar o desvio incondicional.

Anos 1980 e 90

O retorno recente do estilo funcional

Ele volta ao centro do debate a partir dos anos 2010, empurrado por dois fatores práticos: processadores com muitos núcleos e sistemas distribuídos, dois contextos em que estado compartilhado é caro.

O problema que vai atravessar a aula

Uma loja precisa de um relatório de vendas.

A entrada é uma lista de vendas, cada uma com produto, valor e categoria. O relatório precisa considerar apenas as vendas acima de cem reais, aplicar dez por cento de imposto sobre cada uma, e devolver o total por categoria.

É um problema pequeno de propósito, e é representativo: filtrar, transformar e agrupar são as três operações mais comuns em qualquer sistema de informação. O que muda de um paradigma para outro não é o resultado, é a forma como o programa é organizado para chegar até ele.



Os três programas produzem exatamente a mesma saída

A comparação é sobre legibilidade, testabilidade e facilidade de mudança, e não sobre correção.

BLOCO 1

Imperativo e procedural

O estilo mais antigo e mais direto: uma sequência de instruções que modifica o estado passo a passo.



O relatório em estilo procedural

Um laço percorre a lista e um dicionário acumulador é modificado a cada iteração.

PYTHON, ESTILO PROCEDURAL

```
def relatorio(vendas):  
    total_por_categoria = {}  
  
    for v in vendas:  
        if v["valor"] <= 100:  
            continue  
  
        liquido = v["valor"] * 0.9  
        cat = v["categoria"]  
  
        if cat not in total_por_categoria:  
            total_por_categoria[cat] = 0  
        total_por_categoria[cat] += liquido  
  
    return total_por_categoria
```



O que esse estilo tem de bom

É o mais próximo de como a maioria das pessoas descreve um procedimento em português, e é fácil de acompanhar linha a linha com um depurador. Para quem está começando, costuma ser o mais legível dos três.

Onde esse estilo começa a doer

1

Estado mutável compartilhado

O dicionário acumulador muda a cada volta do laço. Se duas partes do programa alterarem a mesma estrutura, descobrir quem causou um valor errado exige percorrer todo o caminho.

2

As três tarefas ficam embaralhadas

Filtrar, calcular e agrupar acontecem dentro do mesmo laço. Testar apenas a regra do imposto exige executar o resto junto, e reaproveitar essa regra em outro relatório exige copiar código.

3

Concorrência fica arriscada

Dividir esse laço entre vários núcleos de processamento significa vários fluxos escrevendo no mesmo dicionário ao mesmo tempo, o que exige mecanismos de sincronização e abre espaço para defeitos difíceis de reproduzir.

Nada disso é motivo para abandonar o estilo

Para um script de vinte linhas que roda uma vez, essas objeções não se pagam. Elas passam a importar quando o código vai ser mantido, testado e reaproveitado.

A armadilha mais famosa do Python

Um exemplo pequeno que mostra por que estado mutável merece desconfiança.

O CÓDIGO QUE PARECE CORRETO

```
def adicionar(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar("a"))    # ["a"]
print(adicionar("b"))    # ["a", "b"]  ?!
print(adicionar("c"))    # ["a", "b", "c"]
```

POR QUE ISSO ACONTECE E COMO CORRIGIR

```
# a lista padrao e criada UMA vez,
# quando a funcao e definida,
# e sobrevive entre as chamadas

def adicionar(item, lista=None):
    if lista is None:
        lista = []
    lista.append(item)
    return lista
```



A lição vale além do detalhe da linguagem

Sempre que um dado sobrevive além do momento em que deveria, o comportamento passa a depender do histórico de execução. O estilo funcional evita essa classe inteira de surpresa ao não alterar dados no lugar.

BLOCO 2

Funcional

Descrever a transformação em vez de descrever os passos. Dados entram, dados saem, nada é alterado no lugar.



Função pura e função impura, lado a lado

A diferença não está na sintaxe. Está em o que a função precisa do mundo para funcionar.

IMPURA, depende e altera o mundo

```
desconto = 0.10      # variavel global
log = []

def calcular(preco):
    log.append(preco)  # altera fora
    return preco * (1 - desconto)

# o resultado muda se alguém
# alterar a variavel global
```

PURA, tudo entra pelos argumentos

```
def calcular(preco, desconto):
    return preco * (1 - desconto)

# mesma entrada, mesma saida, sempre
# testar e chamar e comparar:

assert calcular(100, 0.10) == 90
```



O critério prático para reconhecer uma função pura

Se for possível substituir a chamada da função pelo seu resultado sem mudar o comportamento do programa, ela é pura. Esse teste mental é suficiente na maioria dos casos.

Duas ideias que sustentam o estilo

1

Função pura

Uma função é pura quando o resultado depende apenas dos argumentos e ela não altera nada fora de si.

A consequência prática é grande: uma função pura pode ser testada isoladamente, sem preparar banco de dados, sem simular arquivo e sem ordem de execução. Basta chamar e comparar o retorno.

2

Imutabilidade

Em vez de alterar uma estrutura existente, produz-se uma nova a partir dela.

Parece desperdício, mas elimina uma classe inteira de defeitos: se ninguém altera o dado original, nenhuma parte distante do programa pode corrompê-lo sem que se perceba.

Efeito colateral não é palavrão

Todo programa útil precisa gravar arquivo, chamar rede ou mostrar algo na tela. O estilo funcional não elimina efeitos, ele os concentra em pontos identificáveis, deixando o resto do código puro e testável.

As três operações que resolvem quase tudo

Filtrar, transformar e acumular. Martin Fowler chama a composição delas de collection pipeline.

FILTER, seleciona

```
nums = [1, 2, 3, 4, 5, 6]

pares = [
    n for n in nums
    if n % 2 == 0
]

# [2, 4, 6]
```

MAP, transforma

```
nums = [1, 2, 3]

dobros = [
    n * 2
    for n in nums
]

# [2, 4, 6]
```

REDUCE, acumula

```
from functools import reduce

nums = [1, 2, 3, 4]

soma = reduce(
    lambda a, b: a + b,
    nums, 0
)

# 10
```



Esse vocabulário é o mesmo em quase toda linguagem

JavaScript usa filter, map e reduce. Java usa stream com filter, map e collect. SQL faz o mesmo com WHERE, SELECT e GROUP BY. Aprender esses três verbos uma vez rende em todas as tecnologias seguintes.

O mesmo relatório em estilo funcional

As três tarefas agora aparecem separadas e nomeadas, cada uma testável por si.

PYTHON, ESTILO FUNCIONAL

```
def acima_de_cem(v):  
    return v["valor"] > 100  
  
def aplicar_imposto(v):  
    return {**v, "valor": v["valor"] * 0.9}  
  
def agrupar(acc, v):  
    cat = v["categoria"]  
    acc[cat] = acc.get(cat, 0) + v["valor"]  
    return acc  
  
def relatorio(vendas):  
    relevantes = filter(acima_de_cem, vendas)  
    liquidas = map(aplicar_imposto, relevantes)  
    return reduce(agrupar, liquidas, {})
```



O ganho concreto

A regra do imposto virou uma função de uma linha, que pode ser testada sozinha, reaproveitada em outro relatório e alterada sem tocar no resto. A função relatorio passou a ser uma frase que declara a sequência de transformações.

O mesmo vocabulário em JavaScript

Nada do que foi visto é exclusivo de Python. O bloco de front-end da disciplina vai usar exatamente estes três métodos.

JAVASCRIPT, O MESMO RELATÓRIO

```
const relatorio = (vendas) => vendas
  .filter(v => v.valor > 100)
  .map(v => ({ ...v, valor: v.valor * 0.9 }))
  .reduce((acc, v) => {
    acc[v.categoria] = (acc[v.categoria] || 0) + v.valor;
    return acc;
  }, {});
```



Três verbos, muitas tecnologias

Java usa stream com filter, map e collect. C# usa LINQ com Where e Select. SQL faz o mesmo trabalho com WHERE, SELECT e GROUP BY. Aprender esse vocabulário uma vez economiza tempo em toda tecnologia seguinte.

Um quarto estilo que você já usa sem saber

Nos três estilos vistos até aqui, o programa descreve como chegar ao resultado. Existe uma família em que ele descreve apenas o resultado desejado, e deixa a decisão do como para o sistema.

É o caso do SQL. A consulta declara quais linhas interessam e como agrupá-las, sem dizer em que ordem ler o disco, se convém usar índice ou como fazer a junção. Um componente chamado otimizador decide isso, e frequentemente decide melhor do que a pessoa decidiria.

HTML e CSS seguem a mesma lógica. A folha de estilo declara que o elemento deve ficar centralizado, sem descrever o algoritmo de posicionamento. O navegador resolve.



Por que isso importa para o projeto do semestre

Nas unidades 3 e 4, o mesmo sistema vai misturar quatro estilos: SQL declarativo no banco, objetos no domínio, pipeline funcional nas transformações e código imperativo na interface. Reconhecer cada um evita escrever no estilo errado por engano.

Quando o estilo funcional compensa

Situação	Compensa	Por quê
Transformar coleções de dados	sim	É exatamente o problema que o vocabulário resolve
Regras de negócio que mudam com frequência	sim	Regra isolada em função pura é trocada sem efeito colateral
Processamento paralelo	sim	Sem estado compartilhado, dividir entre núcleos deixa de ser arriscado
Código que precisa de muito teste	sim	Função pura testa com uma chamada e uma comparação
Interação com tela, arquivo ou rede	em parte	Efeitos existem e precisam de um lugar, ainda que concentrado
Algoritmo com muitos passos interdependentes	nem sempre	Encadeamento longo pode ficar menos legível que um laço direto



O exagero também existe

Encadear oito transformações em uma linha só é tão ruim quanto um laço de cinquenta linhas. Legibilidade continua sendo o critério final.

BLOCO 3

Orientado a objetos

Guardar juntos os dados e as operações que fazem sentido sobre eles, e distribuir responsabilidade.



A ideia central

Nos dois estilos anteriores, os dados ficam de um lado e as funções que os manipulam ficam de outro.

A orientação a objetos propõe o contrário: reunir, em uma mesma unidade, os dados e o comportamento que faz sentido sobre eles. Um objeto Venda sabe o próprio valor e sabe calcular o próprio líquido. Ninguém de fora precisa conhecer a fórmula.

O ganho aparece quando o sistema cresce. Se a regra do imposto mudar, existe um único lugar a alterar, e esse lugar é o mesmo que guarda os dados afetados. É o que se chama de coesão, e é o principal argumento a favor do estilo.



É para onde a disciplina vai a partir da próxima aula

As aulas 4 a 8 tratam de classes, encapsulamento, herança, polimorfismo e interfaces. Esta aula apresenta apenas a intuição, para que o conceito não chegue sem contexto.

O mesmo relatório em objetos

A regra do imposto passa a morar dentro da própria venda.

PYTHON, ESTILO ORIENTADO A OBJETOS

```
class Venda:
    IMPOSTO = 0.10

    def __init__(self, produto, valor, categoria):
        self.produto = produto
        self.valor = valor
        self.categoria = categoria

    def e_relevante(self):
        return self.valor > 100

    def liquido(self):
        return self.valor * (1 - Venda.IMPOSTO)

# a venda sabe responder sobre si mesma
```



Repare no que aconteceu com o conhecimento

A alíquota do imposto e o limite de relevância deixaram de estar espalhados pelo relatório e passaram a pertencer à venda. Qualquer parte do sistema que precise dessas regras agora pergunta ao objeto, em vez de reimplementá-las.

Os quatro pilares, em uma frase cada

Cada um deles ganha uma aula inteira na unidade 2. Aqui vale apenas fixar o nome e a intuição.

1 Encapsulamento

O objeto protege o próprio estado e decide o que expõe. Quem está de fora usa a operação oferecida, sem mexer nos dados diretamente.

Aula 5.

2 Herança

Uma classe reaproveita e especializa o comportamento de outra, quando de fato existe uma relação de tipo entre elas.

Aula 6.

3 Polimorfismo

Um mesmo comando produz comportamentos diferentes conforme o objeto que o recebe, sem que quem chama precise saber qual é.

Aula 7.

4 Abstração

A interface declara o que precisa ser feito, e cada implementação decide como. É o que permite trocar uma peça sem tocar no resto.

Aula 7.

A ordem não é decorativa

Encapsulamento sustenta os outros três. Sem estado protegido, herança e polimorfismo viram apenas formas mais elaboradas de espalhar dependência.

Os dois estilos se combinam bem

Objetos guardam as regras. O pipeline funcional as aplica sobre a coleção. Esta é a forma que o projeto do semestre vai adotar.

PYTHON, OBJETOS DENTRO DE UM PIPELINE

```
def relatorio(vendas: list[Venda]) -> dict:
    total = {}
    for v in (x for x in vendas if x.e_relevante()):
        total[v.categoria] = total.get(v.categoria, 0) + v.liquido()
    return total
```

```
# a funcao nao conhece a regra do imposto
# ela apenas pergunta a cada venda
```



A conclusão prática da aula

Python, JavaScript e Java são multiparadigma. A pergunta profissional não é qual paradigma seguir, e sim qual estilo cabe melhor em cada trecho: objetos para representar conceitos do negócio, funções puras para transformar coleções, e código imperativo direto onde a clareza pedir.

Três erros comuns de quem está começando em objetos

Aparecem com regularidade nos projetos, e as aulas 4 a 8 voltam a cada um deles com profundidade.

1

Objeto que só guarda dados

Uma classe com atributos e nenhum comportamento é apenas um dicionário com sintaxe mais longa. Se toda a regra continua fora, nada foi ganho em coesão.

2

Herança usada por preguiça

Herdar para reaproveitar duas linhas cria um acoplamento permanente entre as duas classes. Herança expressa que uma coisa é um tipo de outra, e não que ela empresta código.

3

Uma classe que faz tudo

Uma classe chamada Gerenciador, Utilitario ou Sistema, com quinze métodos sem relação entre si, costuma indicar que a modelagem não encontrou os conceitos reais do problema.

A pergunta que orienta a modelagem

Antes de criar uma classe, vale perguntar que conceito do problema real ela representa. Se a resposta for apenas um agrupamento técnico, provavelmente não é uma classe.

Exercício rápido em dupla

Cinco minutos. Para cada situação, escolher um estilo e justificar em uma frase.

- 1 Converter um arquivo com 200 mil linhas de vendas em um resumo mensal por região.
- 2 Modelar um sistema de reserva de salas, com regras de conflito de horário e de cancelamento.
- 3 Escrever um script que renomeia todos os arquivos de uma pasta seguindo um padrão.
- 4 Calcular, sobre a mesma lista de pedidos, cinco indicadores diferentes que serão exibidos em um painel.
- 5 Buscar, em um banco de dados com milhões de registros, os dez clientes que mais compraram no trimestre.

Respostas defensáveis: 1 funcional, 2 orientado a objetos, 3 procedural, 4 funcional, 5 declarativo com SQL.

Os três estilos lado a lado

Critério	Procedural	Funcional	Orientado a objetos
Onde fica o estado	em variáveis alteradas	não há estado alterado	dentro de cada objeto
Unidade de organização	o procedimento	a função pura	a classe
Facilidade de testar	média	alta	média a alta
Adequado a paralelismo	exige cuidado	natural	depende do desenho
Melhor uso	scripts e algoritmos	transformar dados	modelar o negócio
Risco típico	estado espalhado	encadeamento ilegível	hierarquia complicada



Nenhuma linha diz vencedor

A tabela existe para dar critério de escolha por trecho de código, e não para eleger um estilo para toda a carreira.

Imutabilidade na prática

Python oferece estruturas que não podem ser alteradas depois de criadas. Elas comunicam intenção e evitam alteração acidental.

LISTA, pode ser alterada

```
coords = [10, 20]
coords[0] = 99      # permitido

# se essa lista for passada adiante,
# qualquer funcao pode altera-la
# sem que o chamador perceba
```

TUPLA, não pode

```
coords = (10, 20)
coords[0] = 99
# TypeError: tuple object does
#         not support item assignment

# a estrutura vira uma garantia,
# e nao apenas uma convencao
```



Uma regra prática para o projeto do semestre

Se um dado não deve mudar depois de criado, use uma estrutura que não permita a mudança. É mais barato deixar a linguagem impedir o erro do que confiar que ninguém vai cometê-lo.

Um quinto estilo que aparece no front-end

Na unidade 4 o programa deixa de ter um começo e um fim claros, e passa a reagir ao que o usuário faz.

1

A inversão de controle

Em vez de o programa decidir a ordem das coisas, ele registra funções e espera. Quem decide quando executar é o navegador, a partir de cliques, digitação e respostas de rede.

2

Por que costuma confundir

O código deixa de ser lido de cima para baixo. Duas funções escritas em sequência podem executar em ordem inversa, ou nunca executar, dependendo do que o usuário fizer.

3

O que ajuda a organizar

Manter as funções que reagem a eventos pequenas e delegar a regra para funções puras ou para objetos do domínio. O evento decide quando, o domínio decide o quê.

É por isso que a ordem das unidades é essa

Chegar ao front-end com o domínio já modelado e testado evita misturar dificuldade de interface com dificuldade de regra de negócio.

O paradoxo de Blub

Paul Graham descreveu um efeito que vale como advertência para o resto do curso.

Um programador que só conhece uma linguagem, chamada Blub no ensaio, olha para linguagens mais simples e enxerga o que falta nelas. Mas ao olhar para linguagens mais expressivas, ele não enxerga o que ganharia, porque só percebe a ausência dos recursos que já sabe usar.

Aplicado a paradigmas, o efeito é o mesmo: quem nunca escreveu no estilo funcional não sente falta dele, porque resolve tudo com laços e não percebe o custo disso. A única saída é escrever no outro estilo por algum tempo, mesmo achando estranho no começo. É exatamente o que o laboratório de hoje propõe.

Paul Graham, Beating the Averages, 2001.

Cada estilo levado ao exagero

Todo estilo tem uma versão caricata, e reconhecê-la ajuda tanto quanto conhecer a versão boa.

1

Procedural exagerado

Uma função de trezentas linhas com sete níveis de indentação, alterando meia dúzia de variáveis declaradas no topo. Funciona, e ninguém consegue alterar com segurança.

2

Funcional exagerado

Uma linha só encadeando nove transformações, com funções anônimas dentro de funções anônimas. É elegante para quem escreveu e ilegível para quem chegou depois.

3

Objetos exagerados

Uma hierarquia de seis níveis de herança para representar um cadastro simples, com classes abstratas que existem apenas para satisfazer a arquitetura.

O critério final é sempre o mesmo

Código é lido muito mais vezes do que é escrito. Entre duas soluções corretas, a mais legível para quem vai manter costuma ser a decisão profissional certa.

BLOCO 4

Laboratório

Refatorar do procedural para o funcional, e defender qual das versões deve ir para o projeto.



Refatoração guiada

O ponto de partida é um script procedural que será disponibilizado no repositório da disciplina.

1

Ler e executar o script recebido

Cerca de 60 linhas, com um laço único que filtra, calcula e agrupa ao mesmo tempo. Rodar, entender a saída e escrever em uma frase o que ele faz.

2

Escrever testes antes de mexer

Três testes com pytest, cobrindo o caso comum, o caso de lista vazia e o caso em que nenhuma venda passa do filtro. Refatorar sem teste é apostar.

3

Separar as três responsabilidades

Extrair uma função pura para filtrar, uma para calcular o líquido e uma para agrupar. Os testes precisam continuar passando a cada extração.

4

Compor as funções em um pipeline

Reescrever a função principal como uma sequência de transformações. Comparar o resultado com a versão original.

5

Escrever a justificativa

Um arquivo REFATORACAO.md com as duas versões, o tempo gasto e cinco linhas defendendo qual delas a equipe levaria para o projeto do semestre, com o critério

Entregável: REFATORACAO.md, o script nas duas versões e a suíte de testes passando, com commit em aula

O QUE FICA DESTA AULA

Paradigma é uma decisão sobre onde fica o estado do programa.

- Procedural descreve os passos e altera o estado. É direto e adequado a scripts e algoritmos.
- Funcional descreve transformações sem alterar dados. Rende em coleções, testes e paralelismo.
- Orientado a objetos guarda dados e comportamento juntos. Rende ao modelar conceitos do negócio.
- As linguagens do curso são multiparadigma, e por isso a escolha é do programador e precisa de justificativa.

Próxima aula: Tipagem, execução e memória, agora com profundidade e laboratório de medição

Leitura para a próxima aula: The Floating-Point Guide, seções Basic Answers e Errors. Trazer o computador com Python e Node.js prontos para rodar.



Referências desta aula

LIVROS

SEBESTA, Robert W. Conceitos de Linguagens de Programação. 11. ed. Porto Alegre: Bookman, 2018.

Capítulos 15 e 16 tratam de programação funcional e lógica, com o histórico e a fundamentação dos estilos vistos aqui.

ABELSON, Harold; SUSSMAN, Gerald J. Structure and Interpretation of Computer Programs. 2. ed. MIT Press, 1996.

Clássico da abordagem funcional, disponível gratuitamente na internet. Leitura densa e recompensadora.

RAMALHO, Luciano. Python Fluente. 2. ed. São Paulo: Novatec, 2023.

O capítulo sobre funções como objetos de primeira classe cobre exatamente o ferramental funcional usado nesta aula.

TATE, Bruce A. Seven Languages in Seven Weeks. Pragmatic Bookshelf, 2010.

Sete linguagens de paradigmas diferentes em sete capítulos. O melhor antídoto prático contra o paradoxo de Blub.

ARTIGOS, BLOGS E DOCUMENTAÇÃO

Martin Fowler, Collection Pipeline

<https://martinfowler.com/articles/collection-pipeline/>

Martin Fowler, a operação map explicada

<https://martinfowler.com/articles/collection-pipeline/map.html>

Paul Graham, Beating the Averages, o paradoxo de Blub

<https://paulgraham.com/avg.html>

Python, tutorial oficial, capítulo 9 sobre classes, leitura para a aula 4

<https://docs.python.org/pt-br/3/tutorial/classes.html>

MDN Web Docs, JavaScript, referência de Array e seus métodos

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array